

Lista de exercícios (Sala) — Árvores

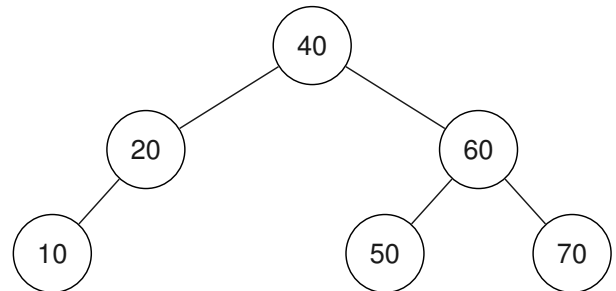
Estrutura base sugerida em C:

```
typedef struct NoArv {  
    int valor;  
    struct NoArv* esq;  
    struct NoArv* dir;  
    int altura;  
} NoArv;
```

1. Classifique cada afirmação em **V** ou **F**.

- I. Toda árvore AVL também é uma ABB.
- II. Em uma ABB, todo nó com valor maior que o valor de seu pai necessariamente pertence à subárvore direita da raiz.
- III. Uma árvore é considerada AVL se a raiz está balanceada.
- IV. O fator de balanceamento de um nó em uma árvore AVL é dado pela altura da subárvore esquerda menos a altura da subárvore direita.
- V. Se o fator de balanceamento de um nó for igual a 2, então esse nó está desbalanceado e é necessário realizar a rotação à direita.
- VI. Em uma árvore AVL, uma rotação é sempre necessária após uma inserção para restaurar o balanceamento.
- VII. Em uma ABB, a travessia em ordem produz os elementos em ordem crescente.
- VIII. Uma árvore com apenas um nó pode ser considerada simultaneamente uma ABB e uma AVL.
- IX. O grau de um nó corresponde ao número de filhos que ele possui.
- X. Em uma ABB sem valores repetidos, o sucessor em ordem de um nó é o seu filho direito.

2. Considere a AVL abaixo.



- (a) Esta árvore está balanceada? Justifique.
- (b) Qual é o sucessor em ordem do nó 40?
- (c) Se o valor 5 for inserido, qual nó ficará desbalanceado? Que rotação será necessária para rebalancear a árvore?

3. Implemente a função:

```
int contarNos(NoArv *raiz){  
    // complete  
}
```

A função deve retornar a quantidade total de nós de uma árvore binária.

4. Implemente a função:

```
int menorABB(NoArv* raiz) {  
    // complete  
}
```

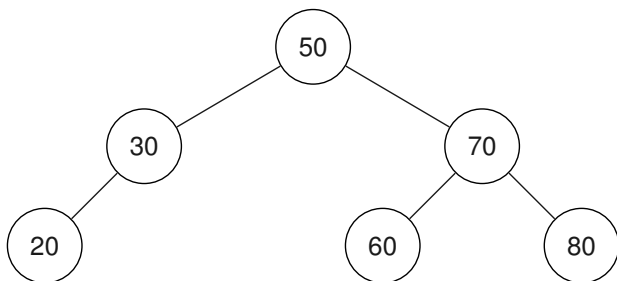
A função deve retornar o menor valor armazenado em uma ABB.

5. Considere a sequência de inserções em uma AVL inicialmente vazia:

42, 18, 9, 27, 63, 57, 71, 54

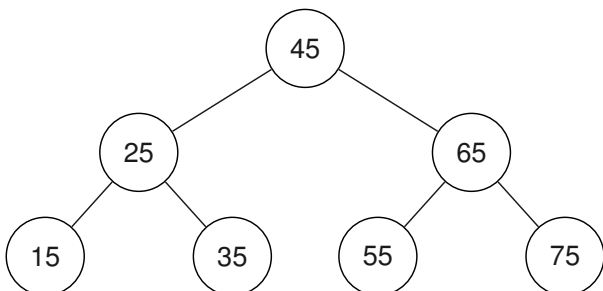
Responda:

- (a) Qual será a raiz da árvore ao final das inserções?
 - (b) Quem será o pai do nó 54?
 - (c) O nó 27 ficará como filho esquerdo ou direito de qual nó?
 - (d) Após construir a árvore final, remova o nó 63. Qual valor deve ocupar o seu lugar na árvore após a remoção?
 - (e) Após a remoção do nó 63, quem passa a ser o pai do nó 71?
6. Considere a árvore AVL abaixo.



Assinale a alternativa correta.

- (a) A inserção do valor 10 exigirá uma **rotação simples à direita no nó 30**.
 - (b) A inserção do valor 65 exigirá uma **rotação simples à esquerda no nó 70**.
 - (c) A remoção do valor 20 exigirá uma **rotação dupla esquerda-direita no nó 30**.
 - (d) A inserção do valor 75 exigirá uma **rotação simples à esquerda no nó 70**.
 - (e) A remoção do valor 80 exigirá uma **rotação simples à direita no nó 70**.
7. Considere a árvore binária abaixo.

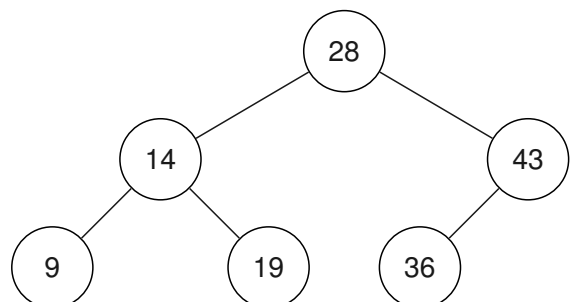


No código de uma ABB, existe uma função de busca com a ideia de percorrer a árvore comparando o valor procurado com o valor do nó atual, retornando o nó quando o valor for encontrado ou NULL caso contrário. Um aluno tentou reescrevê-la da seguinte forma:

```
NoArv* buscarValor(NoArv* raiz, int valor) {
    NoArv* atual = raiz;
    while(atual != NULL) {
        if(valor == atual->valor) {
            return atual;
        } else if(valor < atual->valor) {
            atual = atual->dir;
        } else {
            atual = atual->esq;
        }
    }
    return NULL;
}
```

Responda:

- (a) A função realiza o procedimento de busca corretamente? Justifique.
 - (b) Ao executar `buscarValor(raiz, 35)` na árvore dada, qual será o retorno da função? Mostre o caminho na árvore ao executar a função (quais nós serão consultados).
 - (c) Caso você encontre um erro na função, reescreva a função corrigida e refaça o item anterior.
8. Considere a árvore binária abaixo.



Escreva as sequências de visita para:

- (a) pré-ordem;
- (b) em ordem;
- (c) pós-ordem.